DBMS – ADVANCED SQL QUERY SYSTEM

Objectives:

1. Implement subqueries and nested queries.
2. Find products above the average price.
3. Identify customers with maximum purchases.
4. Generate complex business queries.
5. Prepare advanced SQL reports.

1. Subqueries and Nested Queries

A subquery is a query written inside another SQL query. The inner query is executed first, and its result is used by the outer query.

Example

Find orders whose amount is greater than the average order amount.

```
SQL> SELECT
  2       Order_ID,
  3       Customer_ID,
  4       Total_Amount
  5  FROM Orders
  6  WHERE Total_Amount >
  7       (SELECT AVG(Total_Amount)
  8        FROM Orders);

  ORDER_ID CUSTOMER_ID TOTAL_AMOUNT
---------- ----------- ------------
       201         101         2500
       203         103         3200
```

Explanation

The inner query:

SELECT AVG(Total_Amount) FROM Orders;

calculates the average order amount.

The outer query then displays orders whose Total_Amount is greater than that average.

2. Find Products Above Average Price

Assuming the Product table contains:

- Product_ID
- Product_Name
- Price

```
SQL> SELECT
  2       Product_ID,
  3       Product_Name,
  4       Price
  5  FROM Product
  6  WHERE Price >
  7       (SELECT AVG(Price)
  8        FROM Product);

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------
     PRICE
----------
       102
Moisturizer
       350

       103
Shampoo
       450

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------
     PRICE
----------

       105
Body Lotion
       399
```

2.1 Products Below Average Price

```
SQL> SELECT
  2       Product_ID,
  3       Product_Name,
  4       Price
  5  FROM Product
  6  WHERE Price <
  7       (SELECT AVG(Price)
  8        FROM Product);

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------
     PRICE
----------
       104
Lipstick
       299

       101
Face Wash
       250

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------
     PRICE
----------
```

3. Identify Customers with Maximum Purchases

Step 1: Calculate Customer-wise Purchase

```
SQL> SELECT
  2      Customer_ID,
  3      SUM(Total_Amount) AS Total_Purchase
  4  FROM Orders
  5  GROUP BY Customer_ID;

CUSTOMER_ID TOTAL_PURCHASE
----------- --------------
        101           2500
        102           1500
        103           3200
```

Step 2: Find the Maximum Purchase

```
SQL> SELECT MAX(Total_Purchase)
  2  FROM
  3      (SELECT
  4           Customer_ID,
  5           SUM(Total_Amount) AS Total_Purchase
  6       FROM Orders
  7       GROUP BY Customer_ID);

MAX(TOTAL_PURCHASE)
-------------------
               3200
```

3.1 Display Customer Details with Maximum Purchase

```
SQL> SELECT
  2      c.Customer_ID,
  3      c.First_Name || ' ' || c.Last_Name AS Customer_Name,
  4      SUM(o.Total_Amount) AS Total_Purchase
  5  FROM Customer c
  6  JOIN Orders o
  7      ON c.Customer_ID = o.Customer_ID
  8  GROUP BY
  9      c.Customer_ID,
 10      c.First_Name,
 11      c.Last_Name
 12  HAVING SUM(o.Total_Amount) =
 13      (SELECT MAX(Total_Purchase)
 14       FROM
 15          (SELECT SUM(Total_Amount) AS Total_Purchase
 16           FROM Orders
 17           GROUP BY Customer_ID));

CUSTOMER_ID
-----------
CUSTOMER_NAME
--------------------------------------------------------------------
TOTAL_PURCHASE
--------------
        103
Anitha R
          3200
```

4. Complex Business Queries

Subqueries can be combined with joins and aggregate functions to solve business-related problems.

4.1 Customers Who Placed Orders Above Average Order Value

```
SQL> SELECT
  2      c.Customer_ID,
  3      c.First_Name || ' ' || c.Last_Name AS Customer_Name,
  4      o.Order_ID,
  5      o.Total_Amount
  6  FROM Customer c
  7  JOIN Orders o
  8      ON c.Customer_ID = o.Customer_ID
  9  WHERE o.Total_Amount >
 10      (SELECT AVG(Total_Amount)
 11       FROM Orders)
 12  ORDER BY o.Total_Amount DESC;

CUSTOMER_ID
-----------
CUSTOMER_NAME
--------------------------------------------------------------------
  ORDER_ID TOTAL_AMOUNT
---------- ------------
        103
Anitha R
       203         3200

        101
Harini N
       201         2500

CUSTOMER_ID
-----------
CUSTOMER_NAME
--------------------------------------------------------------------
  ORDER_ID TOTAL_AMOUNT
---------- ------------
```

5. Find Products Whose Sales Are Above Average

```
SQL> SELECT
  2       p.Product_ID,
  3       p.Product_Name,
  4       SUM(od.Subtotal) AS Total_Sales
  5   FROM Product p
  6   JOIN Order_Details od
  7       ON p.Product_ID = od.Product_ID
  8   GROUP BY
  9       p.Product_ID,
 10       p.Product_Name
 11   HAVING SUM(od.Subtotal) >
 12       (SELECT AVG(Product_Sales)
 13        FROM
 14           (SELECT SUM(Subtotal) AS Product_Sales
 15            FROM Order_Details
 16            GROUP BY Product_ID))
 17   ORDER BY Total_Sales DESC;

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------
TOTAL_SALES
-----------
       104
Lipstick
       3200

       101
Face Wash
       3000

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------
TOTAL_SALES
```

6. Customer Purchase Report

```
SQL> SELECT
  2       c.Customer_ID,
  3       c.First_Name || ' ' || c.Last_Name AS Customer_Name,
  4       COUNT(o.Order_ID) AS Total_Orders,
  5       SUM(o.Total_Amount) AS Total_Purchase,
  6       AVG(o.Total_Amount) AS Average_Order_Value
  7   FROM Customer c
  8   JOIN Orders o
  9       ON c.Customer_ID = o.Customer_ID
 10   GROUP BY
 11       c.Customer_ID,
 12       c.First_Name,
 13       c.Last_Name
 14   ORDER BY Total_Purchase DESC;

CUSTOMER_ID
-----------
CUSTOMER_NAME
--------------------------------------------------
TOTAL_ORDERS TOTAL_PURCHASE AVERAGE_ORDER_VALUE
------------ -------------- -------------------
        103
Anitha R
           1           3200                3200

        101
Harini N
           1           2500                2500
```

Report Contains

- Customer ID
- Customer Name
- Number of orders
- Total purchase amount
- Average order value

7. Advanced Product Sales Report

```
SQL> SELECT
  2      p.Product_ID,
  3      p.Product_Name,
  4      COUNT(od.Order_ID) AS Number_of_Orders,
  5      SUM(od.Quantity) AS Total_Quantity_Sold,
  6      SUM(od.Subtotal) AS Total_Sales,
  7      AVG(od.Unit_Price) AS Average_Price
  8  FROM Product p
  9  JOIN Order_Details od
 10      ON p.Product_ID = od.Product_ID
 11  GROUP BY
 12      p.Product_ID,
 13      p.Product_Name
 14  ORDER BY Total_Sales DESC;

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------------
NUMBER_OF_ORDERS TOTAL_QUANTITY_SOLD TOTAL_SALES AVERAGE_PRICE
---------------- ------------------- ----------- -------------
       104
Lipstick
               1                   2        3200          1600

       101
Face Wash
               1                   3        3000          1000

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------------
NUMBER_OF_ORDERS TOTAL_QUANTITY_SOLD TOTAL_SALES AVERAGE_PRICE
---------------- ------------------- ----------- -------------

       103
```

Purpose

• Number of orders for each product

• Total quantity sold

• Total sales

• Average selling price

• Products generating higher sales

8. Category-wise Advanced Sales Report

```
SQL> SELECT
  2      cat.Category_ID,
  3      cat.Category_Name,
  4      COUNT(DISTINCT od.Order_ID) AS Total_Orders,
  5      SUM(od.Quantity) AS Total_Quantity_Sold,
  6      SUM(od.Subtotal) AS Total_Sales,
  7      AVG(od.Unit_Price) AS Average_Price
  8  FROM Category cat
  9  JOIN Product p
 10      ON cat.Category_ID = p.Category_ID
 11  JOIN Order_Details od
 12      ON p.Product_ID = od.Product_ID
 13  GROUP BY
 14      cat.Category_ID,
 15      cat.Category_Name
 16  ORDER BY Total_Sales DESC;

CATEGORY_ID
-----------
CATEGORY_NAME
--------------------------------------------------------------------
TOTAL_ORDERS TOTAL_QUANTITY_SOLD TOTAL_SALES AVERAGE_PRICE
------------ ------------------- ----------- -------------
           4
Makeup
            1                   2        3200          1600

           1
Face Care
            1                   3        3000          1000
```